1. Programming Paradigms

A **programming paradigm** is a fundamental style or "way" of building the structure and elements of a computer program. It's a method of classifying programming languages based on their features.

The four main types of programming paradigms are:

- **Imperative Programming:** This paradigm focuses on describing *how* a program operates. It's composed of a sequence of commands that change the program's state.
- **Declarative Programming:** This paradigm focuses on *what* the program should accomplish without specifying how to do it. The language itself is responsible for figuring out the "how."
- **Object-Oriented Programming (OOP):** This approach revolves around the concept of "objects," which can contain data in the form of fields (often known as attributes or properties) and code in the form of procedures (often known as methods).
- **Functional Programming:** This paradigm treats computation as the evaluation of mathematical functions and avoids changing state and mutable data.

---

2. Procedural vs. Object-Oriented Programming

Feature	Procedural Oriented Programming (POP)	Object-Oriented Programming (OOP)
Approach	Top-down approach.	Bottom-up approach.
Focus	Focus is on procedures or functions.	Focus is on objects and data.
Data Hiding	Does not support data hiding effectively.	Provides data hiding through encapsulation.
Inheritance	Does not support inheritance.	Supports inheritance, allowing code reuse.
Overloading	Does not support function overloading.	Supports method overloading.
Real-world Modeling	Less effective at modeling real-world problems.	More effective at modeling real-world entities.
Example Languages	C, Fortran, Pascal	Java, C++, Python

Export to Sheets

---

3. Object-Oriented Programming (OOP) Features

Here are the core features of OOP with examples in Java:

- **Encapsulation:** The bundling of data (attributes) and methods that operate on the data into a single unit called a class. It restricts direct access to some of an object's components.

Java

```
class Employee {
    private String name; // Data is private

    public String getName() { // Method provides controlled access
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

- **Inheritance:** A mechanism where a new class (subclass or child class) inherits attributes and methods from an existing class (superclass or parent class).

  Java

```
class Animal { // Superclass
    void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal { // Subclass inherits from Animal
    void bark() {
        System.out.println("The dog barks.");
    }
}
```

- **Polymorphism:** The ability of an object to take on many forms. The most common use of polymorphism in OOP is when a parent class reference is used to refer to a child class object.

  Java

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape.");
    }
}

class Circle extends Shape {
    void draw() { // Method overriding
        System.out.println("Drawing a circle.");
    }
}

class Square extends Shape {
    void draw() { // Method overriding
        System.out.println("Drawing a square.");
    }
}
```

- **Abstraction:** The concept of hiding the complex implementation details and showing only the essential features of the object. It is achieved using abstract classes and interfaces.

  Java

  ```
  abstract class Vehicle { // Abstract class
      abstract void start(); // Abstract method (no implementation)
  }

  class Car extends Vehicle {
      void start() { // Implementation provided by subclass
          System.out.println("Car starts with a key.");
      }
  }
  ```

---

4. Benefits of Object-Oriented Programming

1. **Modularity:** OOP systems are modular because they are composed of separate, self-contained objects. This makes programs easier to manage and understand.
2. **Reusability:** Through inheritance, code from existing classes can be reused in new classes, saving time and effort.
3. **Maintainability:** OOP makes code easier to maintain. If you need to modify a program, you can make changes to a specific object without affecting other parts of the program.
4. **Security:** Features like encapsulation and data hiding protect data from unauthorized access, which enhances the security of the program.

---

5. Structure of a Basic Java Program

A basic Java program has the following structure:

- **Package Statement (Optional):** Defines a namespace for classes.
- **Import Statements (Optional):** Imports built-in and user-defined packages.
- **Class Declaration:** Every Java program must have at least one class declaration.
- **Main Method:** The entry point of the program, where the JVM begins execution. `public static void main(String[] args)`
- **Statements:** The code inside the methods that performs the program's tasks.

Java
```
// Package statement (optional)
package com.example;

// Import statement (optional)
import java.util.Scanner;

// Class declaration
public class MyFirstProgram {

    // Main method - the entry point of the program
    public static void main(String[] args) {
```

```
        // Statements
        System.out.println("Hello, World!");
    }
}
```

---

6. Control Statements in Java

Control statements in Java are used to control the flow of execution in a program.

- **Decision-Making Statements:**
  - **if statement:** Executes a block of code if a condition is true.
    - **Syntax:**

      Java

      ```
      if (condition) {
          // statements
      }
      ```

  - **if-else statement:** Executes the `if` block if a condition is true, otherwise executes the `else` block.
    - **Syntax:**

      Java

      ```
      if (condition) {
          // if block statements
      } else {
          // else block statements
      }
      ```

  - **switch statement:** Executes one of many code blocks based on the value of an expression.
    - **Syntax:**

      Java

      ```
      switch (expression) {
          case value1:
              // statements
              break;
          case value2:
              // statements
              break;
          default:
              // statements
      }
      ```

- **Looping Statements:**
  - **for loop:** Executes a block of code a specific number of times.
  - **while loop:** Executes a block of code as long as a condition is true.
  - **do-while loop:** Executes a block of code once, and then repeats the loop as long as a condition is true.

- **Jump Statements:**
  - **break:** Terminates the loop or switch statement.
  - **continue:** Skips the current iteration of a loop.
  - **return:** Exits from the current method.

---

7. `if`, `if-else`, and `switch` Example

Java

```
public class ControlStatementsDemo {
    public static void main(String[] args) {
        int number = 10;

        // if statement
        if (number > 0) {
            System.out.println("The number is positive.");
        }

        // if-else statement
        if (number % 2 == 0) {
            System.out.println("The number is even.");
        } else {
            System.out.println("The number is odd.");
        }

        // switch statement
        int day = 3;
        String dayString;
        switch (day) {
            case 1:  dayString = "Monday"; break;
            case 2:  dayString = "Tuesday"; break;
            case 3:  dayString = "Wednesday"; break;
            default: dayString = "Invalid day"; break;
        }
        System.out.println(dayString);
    }
}
```

---

8. Class, Object, and Access Specifiers

1. **Class:** A **class** is a blueprint or a template for creating objects. It defines the properties (attributes) and behaviors (methods) that all objects of that class will have.
2. **Object:** An **object** is an instance of a class. When a class is defined, no memory is allocated; but when an object is created (instantiated), memory is allocated.
3. **Access Specifiers (Modifiers):** Access specifiers in Java define the accessibility or scope of a field, method, constructor, or class.

---

9. Difference Between a Class and an Object

Basis	Class	Object
Definition	A blueprint or template.	An instance of a class.
Existence	A logical entity.	A physical entity.
Memory	Does not occupy memory.	Occupies memory when created.
Creation	Declared using the `class` keyword.	Created using the `new` keyword.
Example	`Car` is a class.	A specific `Toyota` car is an object.

Export to Sheets

---

10. Encapsulation, Inheritance, Abstraction, and Polymorphism

These are the four fundamental concepts of OOP. Please see the detailed explanation with examples in **question 3**.

---

11. `public`, `private`, and `protected` Access Modifiers

- **public:** A `public` member (class, method, or field) is accessible from any other class anywhere.
- **private:** A `private` member is accessible only within the same class where it is declared.
- **protected:** A `protected` member is accessible within its own package and by subclasses of its class in another package.

---

12. Data Abstraction

Data abstraction is the process of hiding certain details and showing only essential information to the user. It helps in reducing programming complexity and effort.

Abstraction can be achieved in two ways in Java:

1. **Abstract Classes:** An abstract class is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class). It can have both abstract and non-abstract methods.
2. **Interfaces:** An interface is a completely abstract class that can only contain abstract methods and final fields. A class can implement multiple interfaces.

---

13. Data Abstraction vs. Encapsulation

Feature	Data Abstraction	Encapsulation
Focus	Hides implementation complexity.	Hides data.

Feature	Data Abstraction	Encapsulation
Implementation	Achieved using abstract classes and interfaces.	Achieved using access modifiers (`private`, `protected`, `public`).
Purpose	To show only essential features.	To bundle data and methods together.

Export to Sheets

---

14. Role of JVM and Java's Memory Model

The **Java Virtual Machine (JVM)** is an abstract machine that provides the runtime environment in which Java bytecode can be executed. Its main roles are to load, verify, and execute Java bytecode.

Java's Memory Model in the JVM is divided into several areas:

- **Heap:** Stores all the objects and arrays created by the application.
- **Stack:** Stores local variables and partial results for each thread. Each thread has its own private JVM stack.
- **Method Area:** Stores class-level data for every class, such as the runtime constant pool, field and method data, and the code for methods.
- **PC Registers:** Contains the address of the JVM instruction currently being executed.
- **Native Method Stacks:** Contains all the native methods used in the application.

---

15. `switch-case` and `while` loop Example

Java

```
public class SwitchWhileDemo {
    public static void main(String[] args) {
        // switch-case
        int choice = 2;
        switch (choice) {
            case 1:
                System.out.println("You chose option 1.");
                break;
            case 2:
                System.out.println("You chose option 2.");
                break;
            default:
                System.out.println("Invalid choice.");
        }

        // while loop
        int count = 1;
        System.out.println("\nCounting to 5:");
        while (count <= 5) {
            System.out.println("Count: " + count);
            count++;
        }
    }
}
```

Unit 2

---

1. Constructors and `this` Keyword

A **constructor** in Java is a special method that is used to initialize objects. It is called when an object of a class is created.

The `this` keyword is a reference variable that refers to the current object. It can be used to:

- Refer to the current class instance variables.
- Invoke the current class constructor.
- Be passed as an argument in a method call.

Example:

Java
```
class Student {
    String name;
    int rollNo;

    Student(String name, int rollNo) {
        this.name = name; // 'this' refers to the instance variable
        this.rollNo = rollNo;
    }

    void display() {
        System.out.println("Name: " + name + ", Roll No: " + rollNo);
    }
}
```

---

2. `Student` Class with Getters and Setters

Java
```
class Student {
    private int roll;
    private String name;
    private double percentage;

    // Getter for roll
    public int getRoll() {
        return roll;
    }

    // Setter for roll
    public void setRoll(int roll) {
        this.roll = roll;
    }

    // Getter for name
    public String getName() {
        return name;
    }
```

```
    // Setter for name
    public void setName(String name) {
        this.name = name;
    }

    // Getter for percentage
    public double getPercentage() {
        return percentage;
    }

    // Setter for percentage
    public void setPercentage(double percentage) {
        this.percentage = percentage;
    }
}
```

---

3. 2D Array and Row-wise Sum

Java

```
public class TwoDArraySum {
    // Method to calculate and print row-wise sum
    public static void findRowSum(int[][] arr) {
        for (int i = 0; i < arr.length; i++) {
            int sum = 0;
            for (int j = 0; j < arr[i].length; j++) {
                sum += arr[i][j];
            }
            System.out.println("Sum of row " + (i + 1) + ": " + sum);
        }
    }

    public static void main(String[] args) {
        // Declare and initialize a 2D array
        int[][] matrix = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };

        // Pass the array to the method
        findRowSum(matrix);
    }
}
```

---

4. Interfaces and Method Overriding

Java

```
// Interface definition
interface Animal {
    void makeSound(); // Abstract method
}

// Class implementing the interface
class Dog implements Animal {
    // Overriding the method from the interface
    @Override
    public void makeSound() {
        System.out.println("The dog barks.");
    }
}
```

```
class Cat implements Animal {
    // Overriding the method from the interface
    @Override
    public void makeSound() {
        System.out.println("The cat meows.");
    }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        Animal myCat = new Cat();
        myDog.makeSound();
        myCat.makeSound();
    }
}
```

---

5. Primitive vs. Reference Data Types

Feature	Primitive Data Types	Reference Data Types
Definition	Predefined by the language.	Created by the programmer (except for `String`).
Storage	Store the actual value.	Store a reference (memory address) to the object.
Default Value	Have a default value (e.g., `int` is 0).	Default value is `null`.
Examples	`int`, `char`, `boolean`, `double`	`String`, `Array`, `Class` objects

Export to Sheets

---

6. Method Overloading

Method overloading is a feature that allows a class to have more than one method with the same name, as long as their parameter lists are different (either in the number of arguments or the type of arguments).

Java

```
class Calculator {
    // Method to add two integers
    public int add(int a, int b) {
        return a + b;
    }

    // Overloaded method to add three integers
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // Overloaded method to add two doubles
    public double add(double a, double b) {
        return a + b;
    }
}
```

```
public class OverloadingDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println("Sum of 2 ints: " + calc.add(5, 10));
        System.out.println("Sum of 3 ints: " + calc.add(5, 10, 15));
        System.out.println("Sum of 2 doubles: " + calc.add(5.5, 10.5));
    }
}
```

---

7. Command-Line Arguments

Command-line arguments are arguments that are passed to a program when it is run from the command line. In Java, they are passed to the `main` method's `String[] args` parameter.

Java
```
public class CommandLineArgs {
    public static void main(String[] args) {
        System.out.println("Command-line arguments are:");
        for (String arg : args) {
            System.out.println(arg);
        }
    }
}
// To run:
// javac CommandLineArgs.java
// java CommandLineArgs arg1 arg2 "third argument"
```

---

8. Byte Stream and Character Stream

- **Byte Stream:** Used to perform input and output of 8-bit bytes. The main classes are `InputStream` and `OutputStream`. They are suitable for handling binary data like images and audio.
- **Character Stream:** Used to perform input and output for 16-bit Unicode. The main classes are `Reader` and `Writer`. They are suitable for handling text data.

---

9. Predefined Streams in Java

Java has three predefined streams, which are part of the `System` class:

1. **`System.in`:** An `InputStream` for standard input, which is typically the keyboard.
2. **`System.out`:** A `PrintStream` for standard output, which is typically the console.
3. **`System.err`:** A `PrintStream` for standard error output, also typically the console.

---

10. What are command-line arguments in Java?

Please see the detailed explanation and example in **question 7**.

11. `Person` Class

Java

```
class Person {
    private String name;
    private int age;

    // Default constructor
    public Person() {
        this.name = "Unknown";
        this.age = 0;
    }

    // Parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }

    // Getter for age
    public int getAge() {
        return age;
    }

    // Setter for age
    public void setAge(int age) {
        this.age = age;
    }
}
```

12. Method Overloading `add()` Methods

Java

```
class Adder {
    // Method to add two integers
    public int add(int a, int b) {
        return a + b;
    }

    // Overloaded method to add two doubles
    public double add(double a, double b) {
        return a + b;
    }
}
```

13. Variable-Length Arguments (. . .)

Variable-length arguments (varargs) allow a method to accept zero or more arguments of a specified type.

Java
```
public class VarargsDemo {
    // Method with variable-length arguments
    public static int sum(int... numbers) {
        int total = 0;
        for (int num : numbers) {
            total += num;
        }
        return total;
    }

    public static void main(String[] args) {
        System.out.println("Sum: " + sum(1, 2, 3));
        System.out.println("Sum: " + sum(10, 20, 30, 40, 50));
        System.out.println("Sum (no args): " + sum());
    }
}
```

---

14. `Scanner` and `PrintWriter`

Java
```
import java.io.PrintWriter;
import java.util.Scanner;

public class ConsoleIO {
    public static void main(String[] args) {
        // Read input using Scanner
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();
        scanner.close();

        // Write formatted output using PrintWriter
        PrintWriter writer = new PrintWriter(System.out);
        writer.printf("Hello, %s! You are %d years old.\n", name, age);
        writer.flush(); // Flush the writer to ensure output is written
    }
}
```

---

15. Banking System: `BankAccount` Class

Java
```
class BankAccount {
    private String name;
    private long accNo;
    private double balance;

    // Default constructor
```

```
    public BankAccount() {}

    // Parameterized constructor
    public BankAccount(String name, long accNo, double balance) {
        this.name = name;
        this.accNo = accNo;
        this.balance = balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposit successful. New balance: " +
balance);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawal successful. New balance: " +
balance);
        } else {
            System.out.println("Insufficient balance or invalid amount.");
        }
    }

    public void display() {
        System.out.println("Account Holder: " + name);
        System.out.println("Account Number: " + accNo);
        System.out.println("Balance: " + balance);
    }
}

public class BankSystem {
    public static void main(String[] args) {
        BankAccount account1 = new BankAccount("John Doe", 123456789,
1000.0);
        account1.display();
        account1.deposit(500.0);
        account1.withdraw(200.0);
        account1.display();
    }
}
```

---

16. Library System: `Book` Class

Java

```
class Book {
    String title;
    String author;
    String ISBN;
    boolean isIssued;

    public Book(String title, String author, String ISBN) {
        this.title = title;
        this.author = author;
```

```
        this.ISBN = ISBN;
        this.isIssued = false;
    }

    public void issueBook() {
        if (!isIssued) {
            this.isIssued = true;
            System.out.println(title + " has been issued.");
        } else {
            System.out.println(title + " is already issued.");
        }
    }

    public void returnBook() {
        if (isIssued) {
            this.isIssued = false;
            System.out.println(title + " has been returned.");
        } else {
            System.out.println(title + " was not issued.");
        }
    }
}
```

---

17. Sorting Command-Line Arguments

Java

```
import java.util.Arrays;

public class SortArgs {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Please provide integers as command-line
arguments.");
            return;
        }

        int[] numbers = new int[args.length];
        for (int i = 0; i < args.length; i++) {
            numbers[i] = Integer.parseInt(args[i]);
        }

        Arrays.sort(numbers);

        System.out.println("Sorted numbers:");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
// To run:
// javac SortArgs.java
// java SortArgs 5 2 8 1 9
```